

專題名稱：智慧購物導航

指導教授：吳世琳 教授、陳嶽鵬 教授

指導教授簽名： _ _ _ _ _ 教授、 _ _ _ _ _ 教授

專題成員：

B1229013 陳宜伶

B1229020 何思顗

B1229023 林依賢

B1222017 周 庠

中華民國：115 年 6 月 1 日

目錄

一、系統簡介	4
1.1 規格目的	4
1.2 規格範圍	4
1.3 參考文件	5
二、系統概述	6
2.1 系統目標	6
2.2 系統範圍	7
2.3 系統架構	8
2.4 軟/硬體建構項目需求概述	10
2.5 軟/硬體環境	13
2.6 一般限制	15
三、設計內容	17
3.1 軟/硬體建構項目架構	17
3.2 軟硬體組件設計說明	20
1. 導航子系統	20
(1) 導航前端模組 (ArNavigationScreen)	20
(2) 視覺語意辨識模組 (Vision Recognition Engine)	20
(3) 拓樸地圖管理模組 (Map Manager)	21
(4) 路徑規劃演算法模組 (Path Planning Component)	22
2. AI 購物助理子系統	23
(1) AI 對話模組 (AIScreen)	23
(2) 預算管理模組 (BudgetScreen)	24
(3) 飲食分析模組 (IngredientsScreen)	25
3. 登入與設定	26

(1) 身份驗證模組 (LoginScreen)	26
3.3 介面設計說明.....	28
3.4 作業程序設計說明.....	32
3.5 輸入/輸出設計說明.....	34
3.6 其他設計說明.....	37
四、需求制設計之追溯與版本管理	39

一、系統簡介

1.1 規格目的

本規格目的在定義系統的技術實作準則與功能邊界，確保開發過程符合預訂需求：

1. 技術標準化：規範 Kotlin 與 Jetpack Compose 框架，並建立 Groq API 與 ML kit 的串接標準。

2. 資料邏輯定義：明確規範各核心功能的操作流程，確保數據處理的一致性：

(1) 賣場導航：定義外部訓練模型匯入 App 後的通訊介面，規範由影像辨識觸發區域判定，進而產生路徑引導訊號的邏輯。

(2) AI購物助理：分為預算管理、健康飲食、AI 對話。確立預算管理中自動化流程與定義預算的數據映射規則，幫助使用掌握預算狀態。規範健康管理在 OCR 辨識食品成分後，即時過敏原檔案比對的邏輯。定義諮詢服務中 AI 回答使用者各項問題的回答範圍與建立自然語言對話的系統格式，以確保對話邏輯能正確輔助使用者。此外，整合附近搜尋功能以檢索周邊門市資訊與距離，幫助使用者根據當前位置快速選定採購目標。

3. 狀態管理規範：定義基於 JSON 檔案的輕量化資料持久化機制。規範 shopping_list.json、diet_records.json 等檔案的讀寫權限與同步時機，確保預算數據、導覽進度與健康管理在 App 跨頁面導覽時能保持一致性與存取穩定度。

4. 預警機制準則：此規則針對於建立效期偵測與庫存狀態的關聯邏輯。系統須判斷「有效期限」與「食用狀態」的聯集：僅針對「未食用」且「有填寫效期」之品項進行過期預警；當商品狀態標記為已食用（Consumed）時，系統應用即時解除監控，以確保提醒資訊的精確度。

1.2 規格範圍

我們的專題「智慧購物導航」的目的是開發一款以 Android 為基礎的綜合型智慧購物助理應用程式。系統的規格範圍主要涵蓋「賣場導航」、「AI 小助理」兩大應用模組，並明確界定相關的實作邊界與排除項目。

1.系統涵蓋範圍

(1) 主模組：賣場導航與路徑規劃

- 透過手機鏡頭擷取第一人稱視角影像，使用GroundingDINO物件檢測模型以及VLM模型，進行環境語意解析與拓樸地圖的建立。
- 解析使用者的購物清單，將商品分類至對應的地圖區域，並運用演算法規劃最短購物路徑，最後使用 AR 箭頭和文字進行引導。

(2) AI 購物助理

- 使用 ML-Kit 進行購買商品收據掃描與視覺化預算管理功能。此外，還採用了 Paddle OCR 技術，掃描並辨識食品的成分表，進一步達成本地端過敏原比對預警、建議攝取熱量、食物熱量換算運動熱量等健康飲食功能。最後用大型語言模型（LLaMA 3.3）回覆使用者，讓使用者能夠與 AI 進行對話互動。
- 涵蓋整合空間定位技術以即時檢索周邊門市資訊與距離。

2. 系統不涵蓋範圍與限制

- 排除硬體基礎設施建置：本系統不需要提前在賣場內佈置藍芽信標、Wi-Fi 發射器等定位設備，也不需建立詳細的3D 地圖。
- AI 小助理的協助範圍，不提供即時的電商平台比價、賣場實際庫存查詢或線上金流支付功能

1.3 參考文件

- [1] 張睿（2005）。OCR後製處理資料分析與圖形歸檔〔碩士論文，亞洲大學〕。華藝線上圖書館。<https://www.airitilibrary.com/Article/Detail?DocID=U0118-0807200916284776>
- [2] Halama, M., Nowak, S. & Polys, K. Semantic orientation for indoor navigation system using large language models. Sci Rep 15, 41519 (2025).
<https://doi.org/10.1038/s41598-025-25281-0>
- [3] Han, L., Min, H., Hwangbo, G., Choi, J., & Seo, P. H. (2025). DialNav: Multi-turn dialog navigation with a remote guide. arXiv preprint arXiv:2509.12894.<https://doi.org/10.48550/arXiv.2509.12894>
- [4] Yin, H., Xu, X., Zhao, L., Wang, Z., Zhou, J., & Lu, J. (2025). Unigoal:

Towards universal zero-shot goal-oriented navigation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 19057-19066).

- [5] Nie, D., Guo, X., Duan, Y., Zhang, R., & Chen, L. (2025, October). Wmnav: Integrating vision-language models into world models for object goal navigation. In 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (pp. 2392-2399). IEEE.

二、系統概述

2.1 系統目標

本系統的目的在打造一款以 **Android** 為基礎的「智慧購物導覽」應用程式，核心專注於解決大型賣場導航之難題，並輔以 **AI** 購物助理提供全方位的購物支持，提升消費者的購物效率與體驗。

1. 核心目標：導航與路徑最佳化

本系統首要目標在於解決消費者在大型賣場（例如 **IKEA**、**家樂福**）中常遇到的找不到商品所在區域或是為了買齊需要物品繞了遠路等問題。

- 不需硬體設備進行環境定位：透過手機鏡頭擷取影像並辨識環境中的標示牌，運用視覺多模態模型建立拓樸地圖，免除賣場需額外建置藍牙或 **Wi-Fi** 定位硬體的龐大成本。
- 自動化路徑規劃：系統能直接連接使用者的購物清單，運用演算法規劃出最短購物動線，並以擴增實境（**AR**）與文字進行指引，提升消費者的購物效率和體驗。

2. 整合型 **AI** 購物助理

除了核心導航外，系統整合 **Groq API**、**LLaMA 3.3** 大型語言模型與 **ML Kit OCR** 技術，打造一個「**AI** 購物助理」，協助使用者處理導航之外的各項購物需求：

- 健康飲食：系統透過 **Paddle OCR** 光學字元辨識技術，自動擷取食品包裝資訊，提供個人化的成分預警（如過敏原辨識）與熱量運動換算建議，協助做出健康的飲食選擇。

- 預算管理：提供直覺的清單編輯與收據掃描功能，並透過圓環圖（DonutChart）將預算使用狀況視覺化，協助使用者精準掌握財務規劃。
- AI 對話：透過大語言模型（LLaMA3.3），提供使用者能與AI進行對話互動功能。

2.2 系統範圍

本系統以 Android 智慧型手機為主要運作裝置，主要應用範圍限定於賣場環境，系統核心之運作範圍、可用功能與限制如下：

1. 賣場導航：

(1) 視覺語意定位與地圖建構範圍：

- 系統定位機制以電腦視覺、GroundingDINO 物件檢測模型、VLM視覺多模態模型進行環境語意解析，無設置賣場內部的實體定位硬體。
- 地圖的建構機制採用動態拓樸機制，將走過且辨識出的區域紀錄節點與連接邊。
- 僅建立語意拓樸地圖，不進行高精度3D幾何密集建圖，以降低行動裝置的負擔。

(2) 路徑規劃範圍：

- 使用者編輯完購物清單後，系統會透過自然語言處理技術，將使用者輸入的購物清單之項目與地圖節點進行語意匹配，將物品對應至地圖上的區域類別。
- 確定目標節點後，系統將規劃由起點、經各項目標節點、以及前往至結帳區、最後到出口的最短路徑。

(3) 導航指引與安全提示範圍：

- 導航介面採用AR箭頭疊加，並搭配文字輔助，顯示於手機畫面。
- 若因環境光線過暗或人潮遮蔽導致視覺特徵遺失，系統將暫停更新導航方向，直到重新捕捉到有效特徵為止。

2. AI 購物助理：

(1) 預算管理與健康飲食範圍：

- 光學字元辨識(OCR)僅限於掃描實體購物收據與食品包裝之成分標示。
- 過敏原比對與熱量換算皆以本地端使用者設定與系統內建之通用換算邏輯為準。

(2) AI 對話範圍：

- AI 對話範圍限定於購物建議、食材搭配與系統操作問答。
- 不提供線上金流串接支付、賣場實際庫存查詢或電商平台即時比價功能。

2.3 系統架構

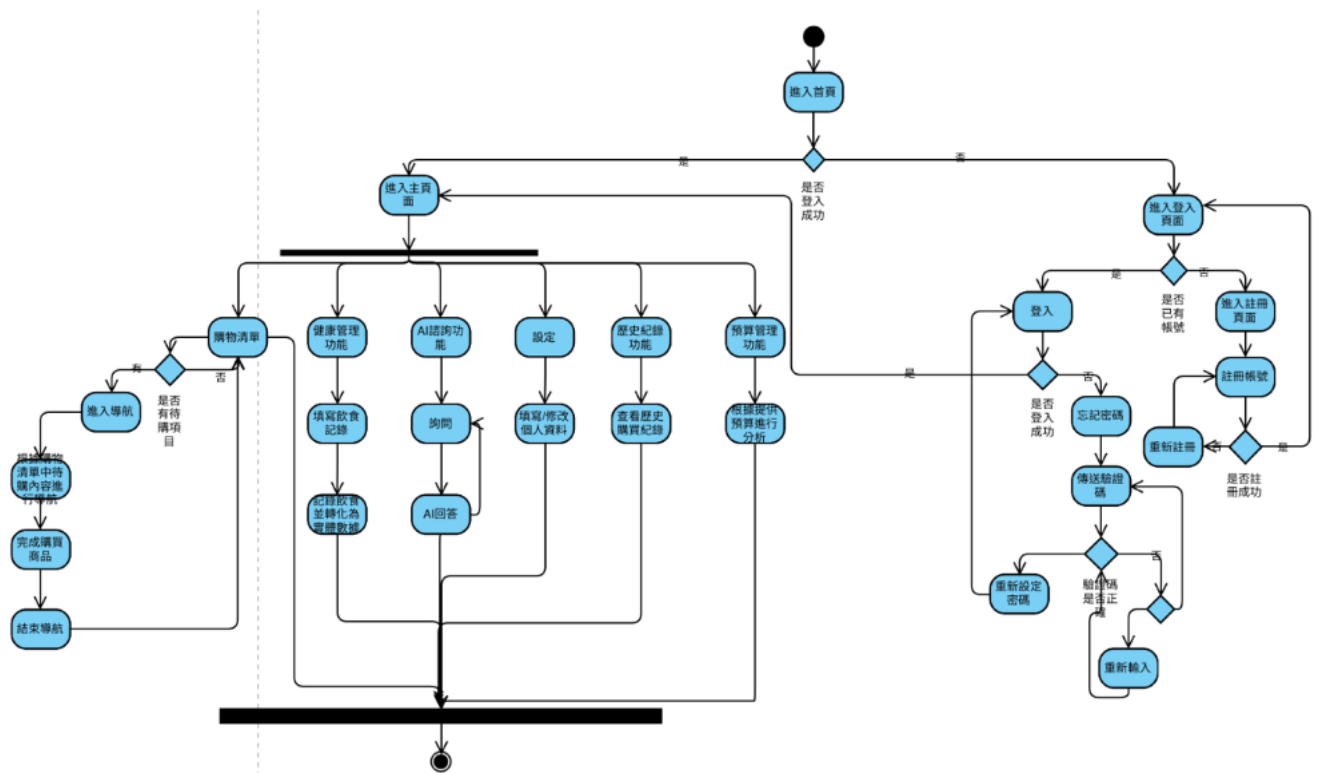


圖 1 系統活動圖

智慧購物導航 — 系統架構

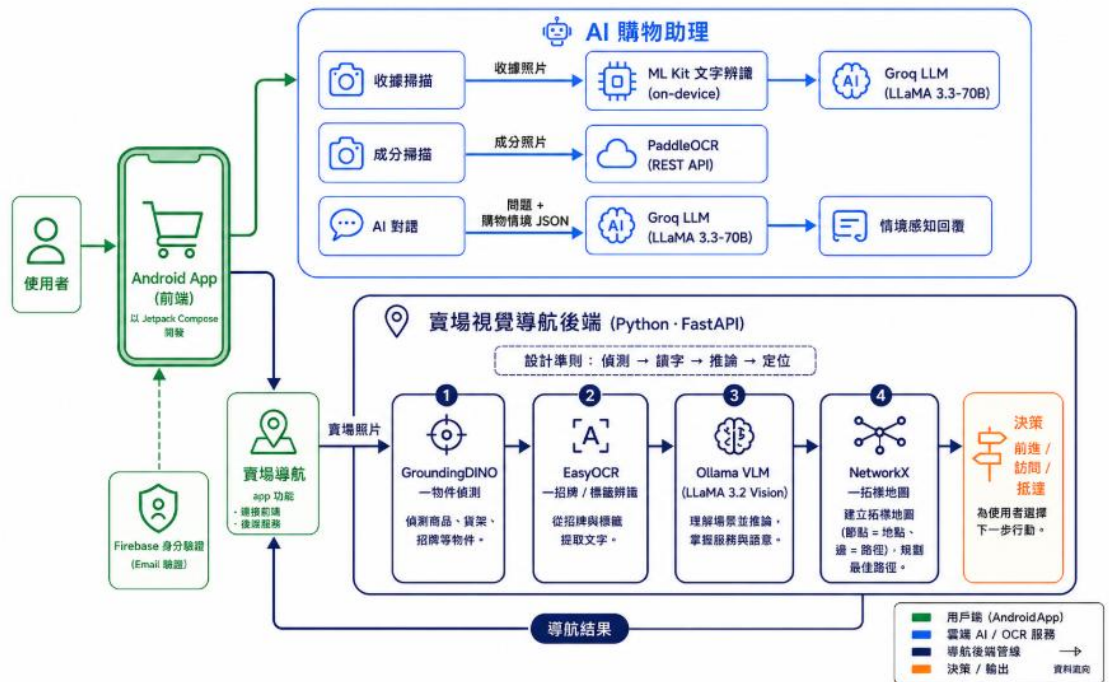


圖 2 系統架構圖軟體分層架構：

採用單一 Activity 架構 (Single Activity Architecture)，區分為以下層級：

1. UI表現層 (Presentation Layer)：

- (1) 利用 Jetpack Compose 構建，透過 NavHost 管理路由頁面。
- (2) 以 MainContainer 作為全域狀態持有者 (State Holder)，負責分發 shoppingItems 與 dietRecords 等狀態。

1. 應用邏輯層 (Application Logic Layer)：

- (1) 拓樸地圖與導航：負責處理拓樸地圖和導航邏輯。結合 GroundingDINO 物件偵測、EasyOCR 文字辨識、VLM 進行環境感知，建立或更新拓樸地圖，最後結合演算法計算多目標的最佳購物路徑。
- (2) AI 購物助理：整合 Retrofit 與 Groq API，執行 LLaMa3.3 模型進行自然語言解析與收據結構化。
- (3) 視覺處理：採用 Camera X 擷取影像，並利用 Paddle OCR 執行即時 OCR 文字辨識。

2. 資料持久層 (Data Persistence Layer)：

- (1) 使用 kotlinx.serialization 進行序列化，將資料儲存於本地 JSON 檔案。
- (2) 將購物清單、健康飲食紀錄與預算數據透過 JSON 格式儲存於本地端。

3. 外部服務整合：

(1) Groq API (LLaMA

3.3)：負責AI助理對話、將收據進行JSON格式化解析。

(2) Firebase

Authentication：負責用於使用者帳號建立與Email身分驗證安全機制。

(3) Google Places API (New)：用於即時門市檢索與空間定位。

4. 硬體與元件：

(1) 相機 (CameraX)：擷取賣場第一人稱影像，供建圖與導航使用。再來則是拍照擷取影像，以用於進行收據OCR分析使用。

(2) 感測器 (Sensors)：結合陀螺儀與加速度計，用於了解使用者行進方向，並輔助 AR 導航箭頭的穩定度。

(3) 網路組件：需具備穩定的 4G/5G 或 Wi-Fi 以即時傳輸影像至雲端模型。

5. 資料結構設計：

(1) 拓樸圖結構：以有向圖形式儲存於 topological_map.png 及 map.json(地圖)，包含節點、邊與路徑權重。

(2) JSON檔案系統：包含 shopping_list.json (購物清單)、diet_records.json (健康飲食紀錄) 與 monthly_budget.txt (預算管理)。

(3) UserProfile (使用者設定)：儲存生理數據 (身高、體重) 與過敏原清單，用於即時健康比對。

2.4 軟/硬體建構項目需求概述

本應用程式以 Kotlin 語言搭配 Jetpack Compose 宣告式 UI 框架開發，整合多項 Android Jetpack 函式庫與第三方 SDK。以下為完整技術堆疊：

類別	技術項目	用途說明
程式語言	Kotlin	主要開發語言，支援協

		程非同步處理。
UI 框架	Jetpack Compose	宣告式介面構建， Scaffold / NavHost 單向 資料流架構。
後端認證	Firebase Authentication	負責帳號管理與電子郵件/密碼登入，含 Email 安全驗證。
AI 推論 (語意對 話)	Groq API + LLaMA 3.3-70b	AI 對話助理、過敏原、 收據結構化解析。
AI 推論 (視覺導 航)	GroundingDINO、EasyOCR、VLM (LLaMA 3.2 vision)	負責導航過程中的實景 環境語意解析與標示牌 辨識。
電腦視覺 OCR	Paddle OCR、ML Kit OCR	食品成分標示辨識與收 據文字擷取。
相機框架	CameraX (Preview + ImageAnalysis)	AR 導航影像擷取(每 3 秒分析一幀)，以及 OCR

		靜態拍照。
網路層	Retrofit 2 + OkHttp + Gson	負責發送 REST API 請求 (Gemini、Groq)與 JSON 解析。
本地儲存	JSON 檔案 (kotlinx.serialization)	輕量化持久層，負責購 物清單、飲食紀錄、月 預算持久化與拓樸地圖 儲存。
圖片載入	Coil (Image Loading Library)	NavigationScreen 中的相 簿圖片顯示
地點與定 位	Google Places API (New)	附近門市即時檢索、空 間定位與距離運算

硬體需求方面，本應用程式需執行於搭載 Android 8.0（Oreo，API Level 26）或以上版本的智慧型手機或平板電腦。裝置需具備以下條件：

- (1) 後置光學鏡頭，用於 OCR 掃描及 AR 導航影像分析使用。
- (2) GPS 定位模組與空間感測器，用於附近門市搜尋與室內定位使用。
- (3) 穩定的網際網路連線，用於支援 Firebase 身份驗證、Google Places API、Groq API 進行呼叫使用。
- (4) 充足的記憶體與運算能力，支援即時推論與 AR 視覺導航指引。

2.5 軟/硬體環境

本系統之環境設定分為開發端(提供開發者進行編碼與調試的環境)與執行端(使用者執行應用程式的環境)。

1. 開發環境

這部分定義在開發系統時使用的軟硬體規格，以確保系統架構的穩定性。

(1) 硬體環境

- 處理器 (CPU)： Intel Core i7 / Apple M2 或同等級以上處理器 (以利運行 Android Emulator)。
- 記憶體 (RAM)： 16GB 以上(確保同時運行 IDE 與 AI 模型測試時不發生延遲)。
- 開發設備： Android 智慧型手機實機(用於 AR 導航與相機功能之調試)。

(2) 軟體環境

- 作業系統： Windows 11 / macOS Sequoia。
- 整合開發環境 (IDE)： Android Studio (Hedgehog 或更新版本)。
- 開發平台： Android 平台，使用 Android Studio 作為開發工具，結合 Android SDK(Target API Level 34)與 Android Jetpack 架構組件進行應用程式開發。

- 開發語言： Kotlin 1.9+ 。
- 版本控制： Git / GitHub 。
- API 測試工具： Postman 或 Insomnia(用於測試 Gemini 與 Groq API 串接)。

2. 執行環境

這部分定義使用者使用「智慧購物導覽」App 所需的最低與建議規格。

(1) 硬體需求

- 終端設備： 搭載 Android 系統的智慧型手機。
- 感測裝置：
 - 後置相機： 至少 1200 萬畫素，需支援自動對焦 (提供 CameraX 擷取清晰標示牌影像)。
 - 感測器： 具備加速度計與陀螺儀 (輔助 AR 導航箭頭之穩定呈現)。
- 網路連線： 需具備穩定的 4G/5G 或 Wi-Fi 連線，以即時呼叫雲端 LMM API 。

(2) 軟體需求

- 作業系統版本： Android 8.0 (Oreo, API Level 26) 以上。
- 必要服務： 需安裝並更新 Google Play Services (以支援 ML Kit

與 Firebase 之運行)。

3. 外部服務

(1) AI： Groq Cloud (LLaMA 3.3)、Gemma 4、LLaMA 3.2 vision。

(2)後端驗證服務： Firebase Authentication。

(3)地點與定位服務： Google Places API (New)

2.6 一般限制

本系統在實際場域運行與佈署時，受限於現有硬體規格、網路環境與外部 API 條件，將面臨以下限制。當處於下述限制情境時，系統之效能、辨識準確度或導航精準度可能會降低甚至難以運作：

1. 實體環境與光學條件限制：

(1) 光照與反光之干擾：系統之 OCR 文字辨識包裝成分，以及影像的視覺特徵擷取，皆依賴影像品質，容易受到光源干擾。若賣場環境光源不足、食品包裝具強烈反光，或招牌字體過小，將大幅降低特徵擷取與文字解析之成功率。

(2) 動態遮蔽限制：若賣場人潮過於擁擠或大型推車遮擋手機相機視線，系統難以從畫面中分析場景資訊，就難以有效辨識出區域節點和貨架特徵，可能導致定位延遲或遺失。

(3) 特徵匱乏限制：若為大型量販店的倉儲區，所見皆是一模一樣無標示的紙

箱或相似的鋼架，或是缺乏顯著的語意特徵或標示牌，視覺模型可能產生誤判或節點混淆。

2. 網路連線與第三方 API 限制：

- (1) API 呼叫頻率限制：系統串接之 Groq API 可能受限於官方之存取頻率限制與配額。若短時間內發送過多影像解析請求，可能面臨伺服器拒絕回應或大幅度延遲。

3. 行動裝置硬體效能限制：

- (1) 算力與記憶體瓶頸：系統需即時處理影像和特徵比對，對於較舊款或記憶體不足的 Android 裝置，可能會出現應用程式強制關閉的狀況。
- (2) 電池續航與過熱：長時間開啟相機畫面並以演算法進行路徑規劃，會造成手機耗電量增加與機身發熱。當裝置進入過熱降頻狀態時，系統的推論速度將受到限制。

三、設計內容

3.1 軟/硬體建構項目架構

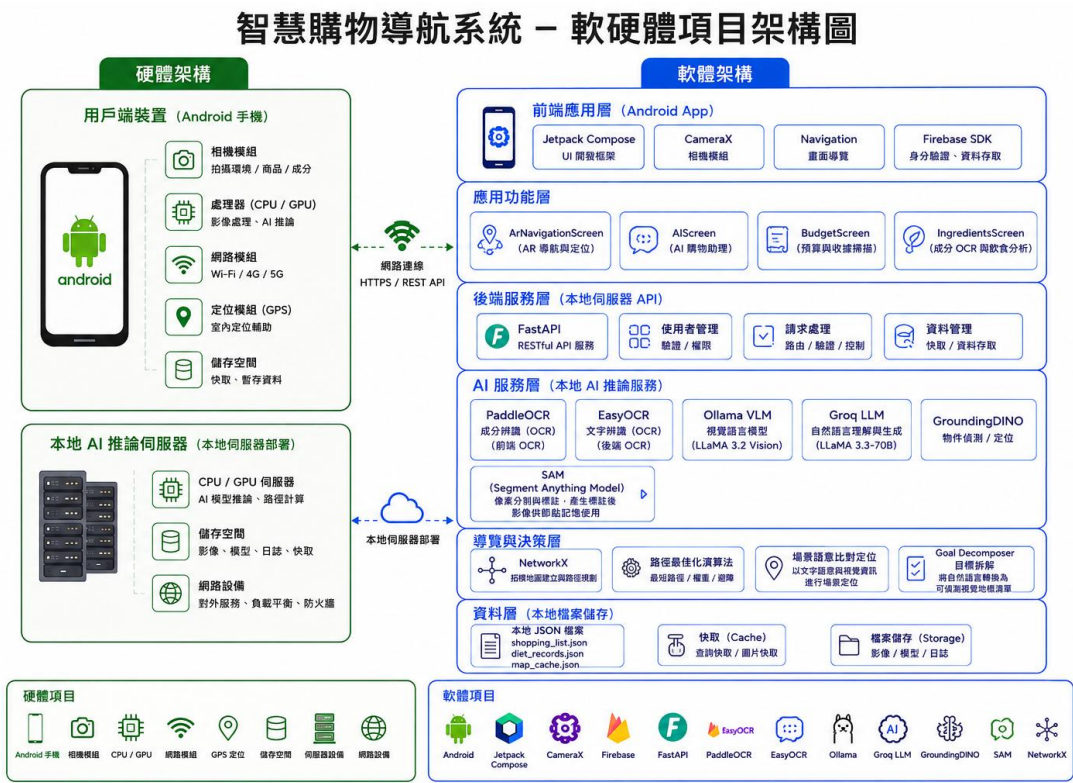


圖3 軟/硬體建構項目架構圖

1. 硬體層配置：

(1) 客戶端（行動裝置）：

使用者端為 Android 智慧型手機，負責即時影像擷取、AR 疊加呈現與使用者互動。裝置須具備攝影機（支援 CameraX）、陀螺儀與加速度計，以維持 AR 導航的視角穩定性。

(2) 伺服器端（推論主機）：後端伺服器需配置至少 8GB VRAM 的 GPU 與 16 核以上 CPU，用於運行 GroundingDINO、SAM 影像分割模型及 Ollama VLM (LLaMA 3.2 Vision 11B) 等模型。

2. 應用程式核心架構（Android App）：

前端以 Kotlin 1.9+ 搭配 Android Jetpack Compose 開發，整體採用單一 Activity 架構，路由與畫面切換由 Jetpack Navigation 統一管理。

(1) 單一 Activity 架構設計：

- 進入點：以 MainActivity 作為唯一的 Android 進入點，負責初始化 Firebase

Authentication實例並設定NavHost。

- 具名路由定義：系統定義具名路由，包含login、main_list、teammate_home、ar_navigation與setting。
- 身份驗證：以login作為固定的起始目的地，確保使用者每次啟動時接需通過驗證，保障數據安全。

(2) 全域狀態與持久化中心：

- 狀態管理與監聽：
 - 全域狀態持有者（State Holder）：MainContainer作為main_list的目的地，於Composable層級統一管理核心資料。
 - 核心資料集合：包含shoppingItems（購物清單）、dietrecords（健康飲食紀錄）、與budgetTotalStr（月預算總額）。
- 輕量級資料持久化：
 - 本地檔案映射：資料分別對應裝置內的shopping_list.json、diet_records.json及monthly_budget.txt三個檔案。
 - 自動化寫入機制：利用LaunchedEffect監聽各狀態變化，在資料異動時自動執行序列化並寫入儲存空間，確保數據不遺失。

(3) 使用者介面與資料流規範

- Scaffold佈局框架管理：
 - 頂部導覽控制：透過CenterAlignedTopAppBar整合設定與AR導航的入口。
 - 底部功能導航：使用NavigationBar管理六個主要功能頁籤（首頁、分析、預算、紀錄、助理）。
- 渲染邏輯與數據流向：
 - 動態內容渲染：藉由selectedTab狀態索引動態決定當前渲染的子畫面Composable。
 - 單向資料流（UDF）實作：子畫面透過參數接收State與回呼（Callback），確保資料流向單一且易偵錯。

3. 後端軟體架構（FastAPI 推論服務）：

後端以 Python Fast API 建立 RESTful API 服務，作為 AI 推論的統一入口。

導航的運作主要有四個步驟：物件偵測、擷取文字、推論、定位。



圖4

導航流程圖

- (1) 系統收到照片後，會用GroundingDINO模型把關鍵物件標出來。
- (2) 透過EasyOCR辨識招牌上的文字。
- (3) 取得文字和影像特徵後，交給Ollama3視覺語言模型進行場景理解和推論。
- (4) 把推論結果帶入 NetworkX建立的拓撲地圖，規劃最佳路徑。

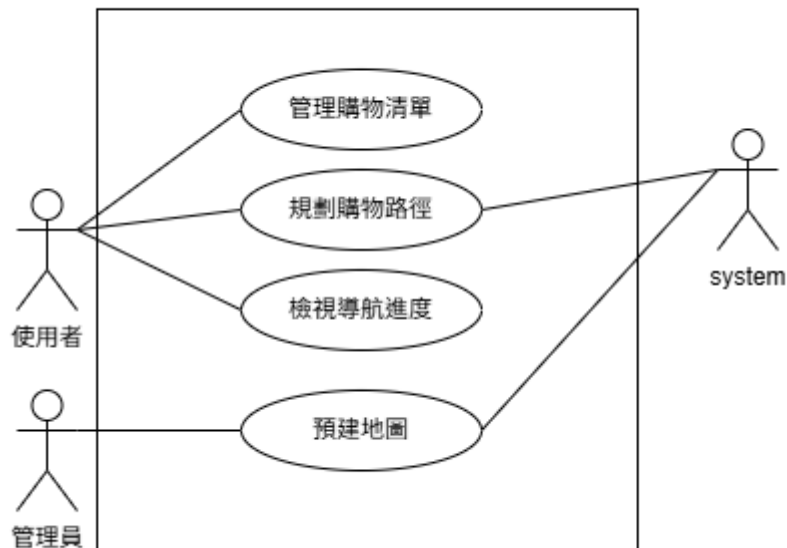
透過這些步驟，系統最後會輸出決策，回傳到前端的App指引使用者移動。

主要的導航推論模組檔案如下：

檔案	職責
server.py	FastAPI 主進入點，定義所有 API 端點，接收前端請求並協調各子模組的呼叫順序
vlm.py	封裝 Ollama LLaMA 3.2 Vision 的推理呼叫，組裝 Prompt 並將模型輸出標準化為 MOVE / ASK / ARRIVED 三種格式
goal_decomposer.py	將使用者自然語言購物品項拆解為 GroundingDINO 可偵測的視覺地標清單

3.2 軟硬體組件設計說明

1. 導航子系統



圖五 導航使用案例圖

(1) 導航前端模組 (ArNavigationScreen)

此模組負責處理導航功能的視覺中心，負責觸發定位循環並呈現導航結果，整合 AR 疊加層與即時影像分析。

- 影像串流管理：利用 CameraX 的 ImageAnalysis 建立動態分析管線，設定採樣頻率（如每 3 秒擷取一幀關鍵影格），並結合 OpenCV 函式庫執行影像前處理，包括透視校正（針對平面圖照片）與影像正規化（針對環境標示牌），以提升辨識精度。
- AR 引導呈現：透過 Compose Canvas 或疊加視窗，將路徑序列轉化為 3D 箭頭或指引標籤。當使用者位置更新時，利用 AnimatedVisibility 動態調整箭頭方向，並結合手機感測器（陀螺儀）修正視角偏差。

(2) 視覺語意辨識模組 (Vision Recognition Engine)

此模組負責將上傳的環境影像轉化為結構化場景語意，作為定位比對與 VLM

推理的輸入基礎。

- 目標拆解：
 - 將使用者輸入的自然語言（例如「找牛奶」），透過 LLM 拆解成感知模組可以辨識的視覺地標清單（例如 ["milk", "dairy", "fridge"]）。
- 視覺感知：
 - GroundingDINO 物件偵測 (GroundingDINO SwinT-OGC)：
GroundingDINO 是一款零樣本 (Zero-shot) 物件偵測模型。當 ArNavigationScreen 擷取或上傳環境影像後，系統會將圖片封裝並發送請求至後端 API。後端透過 GroundingDINO 偵測畫面中的重要物件與環境特徵，例如門口、走道、桌椅、標示物、植物或使用者指定的目標物件。
 - SAM 影像分割與標注：負責在照片中框出門、冰箱、滅火器等實體地標。接收 GroundingDINO 偵測到的物件區域後，進行分割與標注，產生標注後影像，使系統能紀錄每次影像中出現的關鍵物件與場景線索。
 - 環境文字萃取器 (EasyOCR)：負責讀取照片中的文字，讓系統能確認具體的地點。
- 多模態語意推論：將原始影像、偵測結果、使用者目標與歷史紀錄整裡為模型輸入，交由 Ollama VLM 進行多模態分析，輸出目前場景判斷與下一步導航建議。
 - 視覺語言模型(LLaMA 3.2 Vision 11B)：透過本地端的 Ollama 服務運行。它就像系統的大腦，負責接收感知模組的結果（物件與文字）、當前拓撲地圖的歷史摘要，以及最新拍下的照片。
 - 決策輸出 (server/vlm.py)：模型推理後，會標準化輸出三種導航動作之一：MOVE（提供往前或轉彎的具體方向）、ASK（遇到岔路或不確定時向使用者提問）、ARRIVED（已抵達目標）。

(3) 拓撲地圖管理模組 (Map Manager)

此模組負責維護空間的語意拓撲地圖，執行場景比對定位，並管理節點與邊的建立與更新。

- 定位：將視覺語意辨識模組產出的場景語意與 map.json 中所有既有節點的語意標籤進行相似度比對，相似度達閾值則判定使用者位於該節點，

未達閾值則觸發新節點建立。

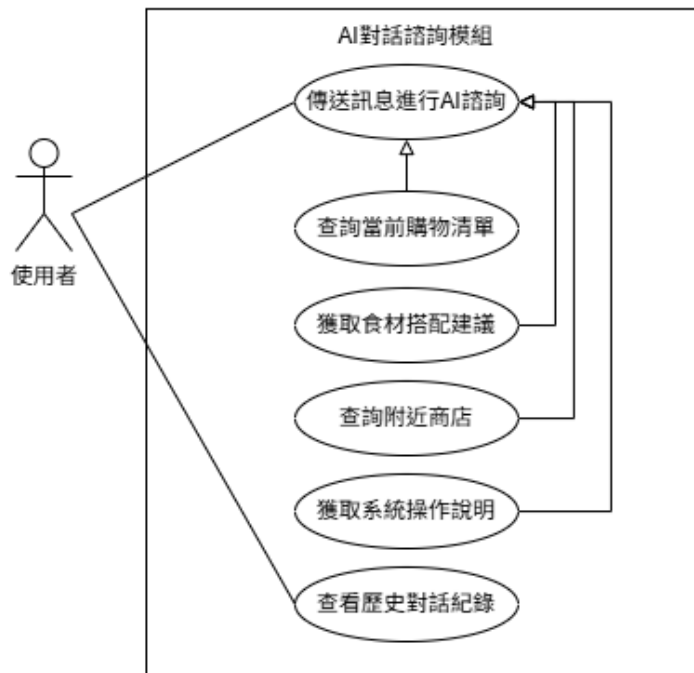
- 建圖：每次影像分析後，系統會將該次觀察所得出的結果建立成一個節點 (Node)，節點內容包含著影像編號、偵測到的物件、模型給予的回覆、使用著標注、標注影像路徑與導航狀態。
- 維護：當使用者依照導航建立移動並再次上傳已向時，系統會在前後兩個節點之間建立邊 (Edge)，表示使用者實際移動路徑，逐步形成拓樸地圖。
- 使用 Python 的 NetworkX 函式庫來建構與管理地圖資料。地圖由兩個基本元素組成：
- 節點 (Nodes)：代表使用者拍攝實景照片的具體位置。每個節點會儲存：
 - 照片的檔案路徑。
 - 由 GroundingDINO 偵測到的物件清單 (如冰箱、門)。
 - 由 EasyOCR 辨識出的環境文字 (如「消防栓」)。
 - 由 VLM 針對該地點產生的文字摘要。
 - 時間戳記。
- 邊 (Edges)：代表使用者從 A 節點移動到 B 節點所採取的「導航動作」。邊上會帶有一個動作標籤 (例如：「右轉並走到這排長廊的盡頭」)。

(4) 路徑規劃演算法模組 (Path Planning Component)

負責以拓樸地圖為基礎，依據使用者購物清單計算從入口經所有目標到結帳區的最短訪問路徑。輸入預建的拓樸地圖，接收使用者的多個購物目標，計算從入口、拜訪所有目標、最終抵達結帳區的最短路徑，並將導航指令傳給前端顯示。

- 目標區域定位：輸入使用者購物清單後，利用自然語言處理，將多項商品分別比對各區域中的物件標籤是否包含目標關鍵字，取信心分數最高的區域為該商品的所在之目標區域，讓每個商品都能匹配到地圖節點。
- 多目標排序計算：確定所有目標節點後，運用開放式旅行推銷員問題 (Open TSP) 邏輯，計算出最佳的拜訪順序，安排使用者從「入口」出發，依序經過各目標區域，最終抵達「結帳區」，以最短總路徑完成拜訪。
- 逐段路徑搜尋：確認目標節點的拜訪順序後，針對兩兩節點之間的具體路線，採用 A*演算法進行細算。

2. AI 購物助理子系統



圖六 AI 購物助理使用案例圖

(1) AI 對話模組 (AIScreen)

- 系統角色：繁體中文購物助理。
- 指定模型：LLama-3.3-70b-versatile。
- 以 Kotlin Coroutines (rememberCoroutineScope) 管理非同步呼叫。
- 輸入：使用者輸入要詢問的訊息。
- 處理流程：使用者送出訊息後，系統將系統提示與使用者輸入封裝為 GroqRequest。透過 Retrofit 建構的 GroqApiService 介面發送 HTTP POST 至 <https://api.groq.com/openai/v1/chat/completions>。
- 回應解析自 choices[0].message.content，並以 (userMsg to aiMsg) 的 Pair

附加至 chatHistory 狀態清單，觸發介面重組。

(2) 預算管理模組 (BudgetScreen)

- 此模組整合整合 OCR 與 Groq API，實現收據解析與結構化處理。
- 輸入：使用者透過相機拍攝或相簿之收據影像。
- 處理流程：
 - 系統透過 `ActivityResultContracts.TakePicture` (相機) 或 `PickVisualMedia` (相簿) 取得影像。
 - 透過 `decodeUriToBitmap` 將 Uri 解碼為 `Bitmap`。
 - 使用 `Paddle OCR TextRecognition` 對其執行光學字元辨識以取得收據中的原始文字。
 - 將原始文字與指定以 JSON 格式輸出的系統提示一起送至 Groq API。
 - 設定 `response_format` 設定為 `json_object`，以確保回應為合法 JSON。
 - 將回傳結果反序列化為 `TidiedReceiptResponse` 資料類別。
- 輸出：系統將解析後的 `line_item` 轉換為 `ShoppingItem` (`isChecked = true`)，並自動整合至主購物清單中，完成消費紀錄的建立與更新。

(3) 飲食分析模組 (IngredientsScreen)

- 影像擷取與 Paddle OCR 辨識：
 - 影像暫存機制：使用 FileProvider 建立名為 ocr_scan.jpg 的暫存檔。
 - 拍攝與輸入建構：透過 TakePicture 合約拍攝後，利用 InputImage.fromFilePath 建立 Paddle OCR 專用隻 InputImage。
 - 字元辨識處理：以 TextRecognition.getClient 進行 OCR 運算，並將結果去除換行符後填入 ingredientText 供使用者核對。
- 安全性比對與健康分析邏輯：
 - 即時過敏警示：系統將辨識結果與 UserProfile 中的過敏原清單進行即時比對。
 - 視覺預警回饋：若偵測到相符之過敏成分，將透過 AnimatedVisibility 觸發紅色警告顯示。
 - 熱量建議與換算：依據使用者個人資料屬性（性別與年齡）查表計算建議熱量。
 - 運動消耗計算：同工將輸入熱量除以特定運動 MET 值，以換算所需運動分鐘數之功能。
 - 資料持久化：將分析結果封裝為 DietRecord 類別，並序列化存入本地 diet_records.json。

3. 登入與設定

(1) 身份驗證模組 (LoginScreen)

- 多模式驗證架構：
 - Firebase 整合：系統深度整合 Firebase Authentication 服務。
 - 雙模式介面：以單一 Composable 支援登入與建立帳號，並透過 `isSignUpMode` 狀態進行切換。
- 驗證流程控制：
 - 信箱驗證機制：登入成功後會檢查 `user.isEmailVerified` 屬性。
 - 攔截邏輯：若電子郵件尚未驗證，系統將呼叫 `auth.signOut()` 並限制其進入主程式。
 - 自動寄送通知：帳號建立完成後，系統會自動觸發 `sendEmailVerification` 寄送驗證信函。

(2) 設定模組 (SettingsScreen)

此設定模組 (`SettingsScreen`) 主要提供以下兩大區塊功能：

- 個人資料編輯：
 - 性別單選按鈕
 - 生日選取 (`DatePickerDialog`)
 - 身高與體重文字欄位

- 過敏原輸入
- 儲存空間管理：
 - 系統透過 `Formatter.formatFileSize` 顯示應用程式內部儲存空間
(`context.filesDir` 與 `cacheDir`) 的實際占用大小，並提供一鍵清除快取功能。
 - 當使用者執行清除操作時，系統呼叫 `deleteRecursively` 方法遞迴刪除快取目錄，以釋放儲存空間。

3.3 介面設計說明



圖七 登入界面 / 註冊介面



圖八 首頁



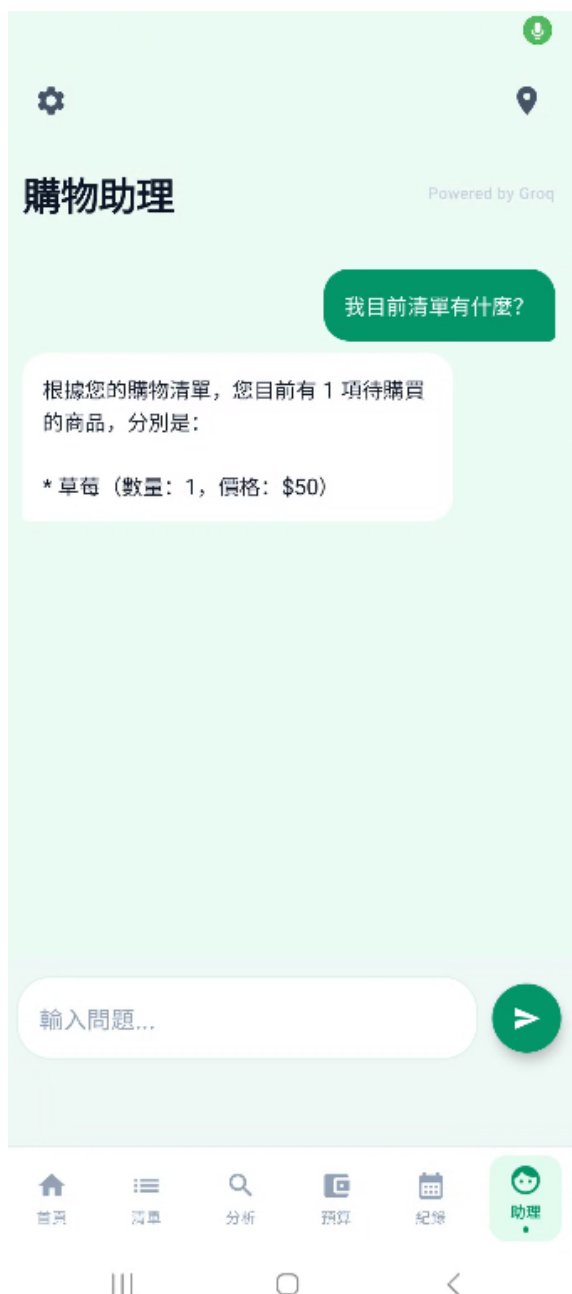
圖九 清單



圖十 預算分析



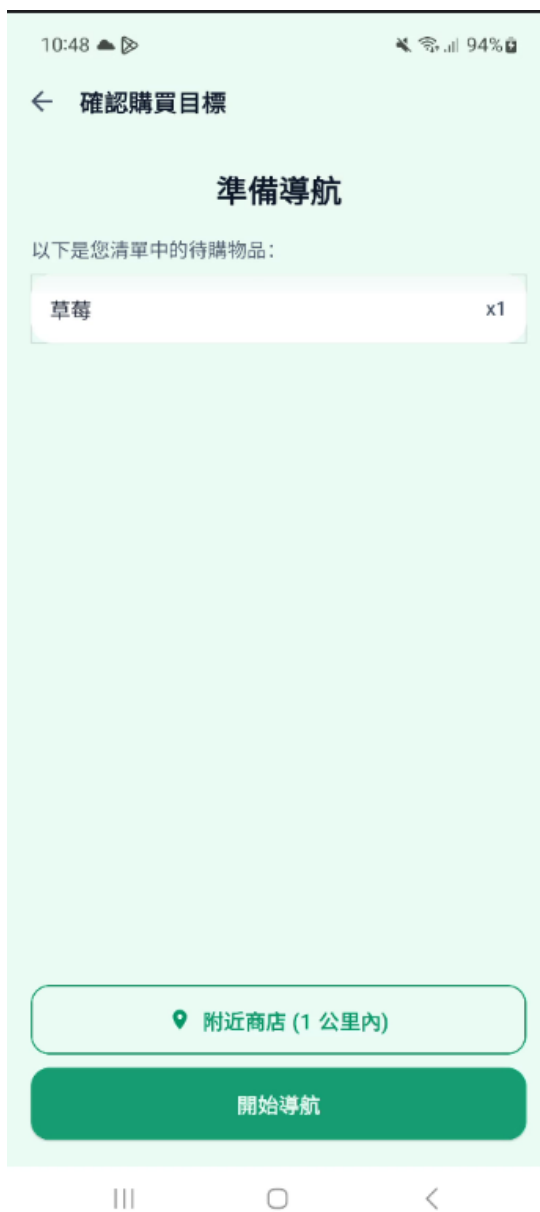
圖十一 健康管理



圖十二 AI 助理



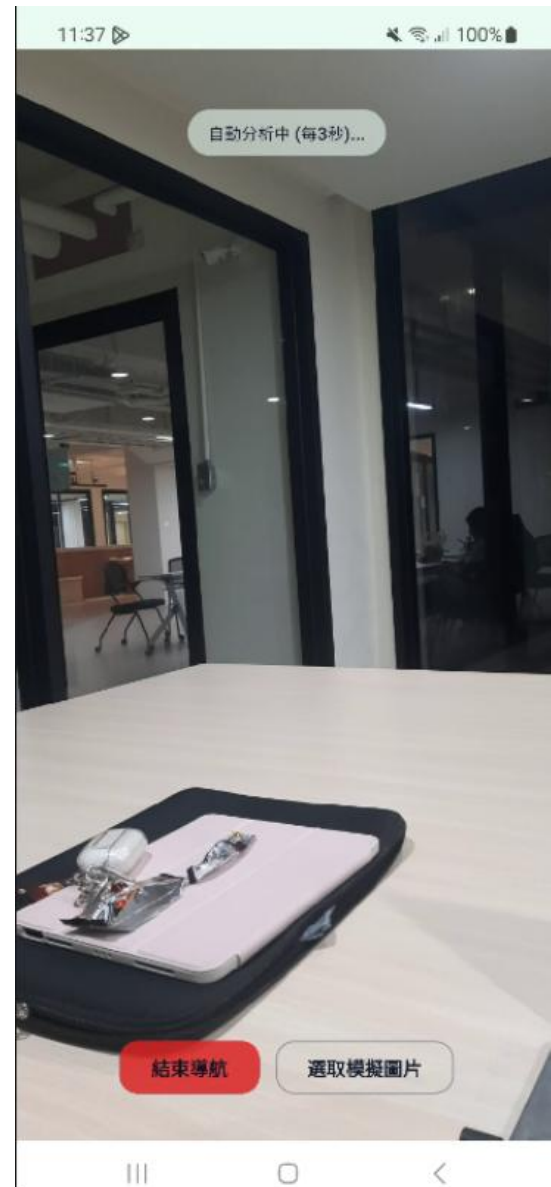
圖十三 個人資料



圖十四 導航開始頁面



圖十五 附近店家搜尋



圖十六 導航開始

3.4 作業程序設計說明

作業程序設計如下：

- (1) 使用者驗證程序：啟動後由 Firebase Auth 檢查登入狀態與 Email 驗證結果，未符合者攔截於登入頁面。
- (2) 食用記錄與分析程序：

- ⑩ 呼叫相機/相簿介面獲取 Bitmap。
- ⑩ 執行 PaddleOCR 擷取文字訊息。
- ⑩ 比對使用者 UserProfile 中的過敏原設定。
- ⑩ 若偵測到衝突則觸發 AnimatedVisibility 紅色預警;確認後將紀錄寫入
diet_records.json。

(3) 效期預警作業邏輯：

- ⑩ 檢查商品條目中是否包含有效期限。
- ⑩ 若有填寫且商品狀態為「庫存中」，則計算當前日期的差距並適時推
播提醒。

(4) AI 對話程序：使用者送出訊息後，啟動分同步協程 (outline) 向

Groq 伺服器發送請求，獲取回應後更新 chatHistory 狀態。

(5) 路徑規劃程序：

- 物品輸入：使用者透過文字於介面輸入預計購買之商品。
- 特徵匹配：將輸入的商品名稱與拓樸地圖之節點進行語意相似度計算
與歸類。
- 路徑運算：確認所有目標節點後，使用演算法規劃最佳路徑，以使用
者進入之入口為起點、以結帳區為終點，計算出一條經過所有目標節
點的最佳訪問順序。

(6) 地圖建立程序：

- 影像擷取：開啟建圖模式後，系統透過手機相機的第一人稱視角擷取影像。
- 語意解析：將關鍵影格傳送給物件偵測和 VLM 模型，解析畫面中的環境特徵。
- 拓樸比對與建圖：
 - 若解析出之語意特徵為已知節點，系統即更新使用者當前位置。
 - 若為未知特徵，系統則將其建立為新的物件節點，並與前一節點相連，達成動態建圖。

3.5 輸入/輸出設計說明

本系統透過多模態輸入擷取資料，並經由處理後轉化為視覺化的管理資訊，

詳細設計如下表：

類別	輸入項目 (Input)	處理與輸出結果 (Output)
影像識別	相機擷取或相簿選取之食品包裝、收據影像 (Bitmap/Uri)。	1. OCR 文本：經 Paddle OCR 提取原始字串。 2. 結構化清單：經 Groq 解析後之 JSON 商品條目。

		3. 動態警示：偵測到有使用者的過敏原時會有紅色 UI 給予提示。
互動諮詢	使用者於 AIScreen 鍵入自然語言。	1. AI 建議：由 LLaMa3.3 生成之食材建物或購物指引。 2. 歷史對話：更新至 chatHistory 清單並渲染於對話筐。
本地數據	儲存裝置內部空間之 JSON 檔案於 TXT 文本。	1. 預算圖表：將預算數據轉化為圓環圖 (Donut Chart) 顯示。 2. 記錄清單：載入並排列飲食足跡與購物清單。
使用者設定	生理數據 (身高/體重/性別)、過敏原清單、月預算總額。	1. 建議數值：計算 BMI 與建議攝取熱量。 2. 換算結果：將熱量攝取轉換為運動分鐘數 (依 MET 值計算)

系統通知	系統時鐘與 is_consumed 狀態位元。	效期提醒：針對「庫存中」商品，若接近有效期限則發出介面或推播通知。
視覺導航 與建圖	相機連續擷取之賣場實景影像 幀	拓樸地圖更新：經 Gemini API 語意解析後，動態建立之節點與連通邊。

拓樸地圖與導航功能，詳細輸入輸出設計如下：

處理階段	輸入/輸出類型	詳細說明
環境感知	輸入	影像串流輸入：透過 Android CameraX，擷取影像格式幀，經系統壓縮轉換為 640x640 解析度之 Bitmap 影像。 手機感測器：結合陀螺儀與加速度計，了解方向，並輔助導航 AR 箭頭。
雲端語意推 論	輸入	發送推論請求，包含處理後的實景影像、系統提示詞，要求模型辨識出賣場特徵標示。
雲端語意推 論	輸出	指定以 JSON 格式結構化回應語意節點名稱、信心水準、相對位置，用於動態更新

		拓樸圖結構。
路徑運算與 決策	輸入	全域拓樸地圖物件，以及經 NLP 語意匹配 後轉換為節點的目標購物清單陣列。
路徑運算與 決策	輸出	A*演算法結果：計算並輸出帶有順序性之 最佳路徑節點陣列。
使用者導引	輸出	視覺引導標示：於相機預覽畫面上繪製 2D 浮動方向箭頭。
使用者導引	輸出	文字輔助標示說明：顯示文字於畫面中， 輔助說明下一步的引導動作。

3.6 其他設計說明

除了核心邏輯功能外，系統在穩定性、資源管理與安全性方面具備以下設計：

(1) 異常處理與容錯機制 (Error Handling)

- ⑩ 網路請求容錯：針對 Groq API 或 Firebase 驗證請求，使用 try-catch 捕捉 IOException 與 HttpException。當網路斷開時，系統將跳出 Snackbar 提示，並允許使用者繼續操作已載入至內存的本地數據，確保 App 不會因 API 逾時而崩潰。

- ⑩ OCR 修正流程：若因影像模糊導致 Paddle OCR 識失敗或資訊不全，系統提供「手動編輯」欄位，讓使用者在資料寫入 JSON 前可自行修正，確保持久化數據的精確性。

(2) 資源回收與效能優化 (Resource Management)

- ⑩ 影像快取清理：系統透過 FileProvider 處理 OCR 掃描所需的暫存圖檔 (如 ocr_scan.jpg)。為避免佔用過多磁碟空間，每次掃描任務結束或於設定頁面點擊「清除快取」，將執行 deleteRecursively 遞迴刪除快取目錄。
- ⑩ 非同步效能控制：所有耗時操作 (如網路 API 呼叫、JSON 序列化、Bitmap 解碼) 均限定在 Dispatcher.IO 或 Default 協程環境執行，避免阻塞主線程 (Main Thread) 造成 UI 凍結。

(3) 隱私權與資料安全(Privacy & Security)

- ⑩ 資料本地化：使用者的敏感生理數據 (如身高、體重) 與飲食隱私僅存儲存於本地 fileDir 下的加密檔案系統中，除必要的帳號同步外，不主動上傳至外部分析平台。
- ⑩ 身份驗證屏障：整合 Firebase Email 驗證機制。若使用者登入後尚未完成信箱驗證，系統將自動呼叫 signOut()並強制攔截於登入介面，確保個人化紀錄僅能被本人存取。

(4) 導航狀態迷失機制

- 視覺特徵遺失處理：當使用者將手機放下、鏡頭被遮蔽，或是於缺乏標示牌的高重複性貨架區時，系統若連續無法從 Gemini API 取得有效語意特徵，將暫停更新相對方向箭頭，並於畫面上提示「請稍微抬高手機環視四周標示」，避免給出錯誤指引。

四、需求制設計之追溯與版本管理

需求編號	需求說明	ArNavigation Screen	Vision Recognition Engine	Map Manager	Path Planning Component	Goal Decomposer	AI Screen	BudgetScreen	IngredientsScreen	LoginScreen	SettingsScreen
F-01	系統須透過手機鏡頭擷取第一人稱視角影像	✓									
F-02	系統須以 GroundingDINO 偵測場景物件，以 EasyOCR 萃取環境文字		✓								
F-03	系統須以視覺語意比對判斷使用者目前位置（無需實體定位硬體）		✓	✓							
F-04	系統須動態建立與維護語意拓樸地圖（節點與邊）			✓							
F-05	系統須將購物清單品項與地圖節點進行語意匹配				✓	✓					
F-06	系統須以 TSP 與 A* 演算法規劃最短購物路徑				✓						
F-07	系統須以 AR 箭頭與文字於畫面上顯示導航指引	✓									
F-08	系統須提供繁體中文 AI 購物諮詢對話功能						✓				
F-09	系統須掃描實體收據並結構化解析為消費紀錄							✓			
F-10	系統須掃描食品成分表並進行過敏原比對與熱量換算								✓		
F-11	系統須提供效期偵測與庫存預警通知								✓		✓
F-12	系統須以 Firebase Email 驗證進行身份驗證，未驗證者禁止進入主程式									✓	
F-13	系統須提供個人資料編輯（性別、生日、身高、體重、過敏原）										✓
F-14	系統須提供儲存空間管理與快取清除功能										✓
F-15	系統須以 Google Places API 檢索附近門市資訊	✓									
NF-01	系統不得因 API 逾時或網路斷線造成應用程式崩潰						✓	✓	✓		
NF-02	使用者生理數據僅儲存於本地，不上傳至外部平台									✓	✓
NF-03	所有耗時操作須於背景執行緒處理，不阻塞主畫面	✓					✓	✓	✓		

需求與設計追溯圖